



SmartSenseCNC — Machine State Detection using Machine Learning

Project Overview

This project presents an intelligent CNC machine monitoring and machine-state classification system using Machine Learning techniques. The system utilizes sensor-based operational data collected from CNC machine environments to identify different machine states in real time.

The dataset used in this project contains machine operational parameters including:

- Machine ID
- Current Consumption (A)
- Vibration (m/s^2)

Using these parameters, the machine is classified into the following operational states:

- MACHINE OFF
 - MACHINE ON (Idle)
 - MACHINE ON + WORKING
-

Objective

The primary objective of this project is to analyze machine sensor behavior and develop a Machine Learning model capable of accurately classifying CNC machine operational states.

The project also focuses on:

- analyzing sensor patterns,
 - understanding machine-state behavior,
 - identifying noisy and overlapping sensor regions,
 - comparing multiple Machine Learning algorithms,
 - selecting the most suitable model for deployment.
-

Dataset Description

The dataset used in this project is based on real sensor observations obtained from the developed prototype system and expanded using controlled synthetic augmentation techniques. Realistic noise, fluctuations, overlapping operational ranges, and outlier conditions were intentionally included to simulate practical industrial environments.

Features Used

Feature	Description
machine_id	Identifier for CNC machine (1-5)
current_A	Measured machine current in amperes
vibration_ms2	Measured vibration in m/s ²
state	Machine operational state

Machine State Logic

MACHINE OFF

- Low current
- Low vibration
- Represents inactive machine condition

MACHINE ON (Idle)

- Current above operational threshold
- Low or moderate vibration
- Represents powered machine without active machining operation

MACHINE ON + WORKING

- Current above threshold
 - High vibration values
 - Represents active machining operation
-

Workflow of This Notebook

This notebook contains the complete AIML workflow for CNC machine-state prediction:

1. Data Loading and Preprocessing
 2. Exploratory Data Analysis (EDA)
 3. Sensor Distribution Analysis
 4. Outlier and Noise Analysis
 5. Correlation and Feature Analysis
 6. Machine-wise Operational Analysis
 7. Machine Learning Model Training
 8. Model Comparison and Evaluation
 9. Confusion Matrix and Performance Metrics
 10. Final Model Selection and Saving
-

Machine Learning Models Used

The following models are evaluated and compared:

- Logistic Regression
 - K-Nearest Neighbors (KNN)
 - Decision Tree
 - Support Vector Machine (SVM)
 - Random Forest
-

Expected Outcome

The final system is expected to:

- accurately classify CNC machine states,
 - handle noisy industrial sensor data,
 - differentiate idle and active machining conditions,
 - support intelligent machine monitoring applications.
-

Tools and Technologies

- Python
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - Scikit-learn
 - Google Colab
-

Project Type

Industrial IoT + Machine Learning based CNC Machine Monitoring System

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
```

```
In [2]: df = pd.read_csv("cnc-dataset.csv")
```

```
In [3]: df.shape
```

```
Out[3]: (6000, 4)
```

```
In [5]: df.head()
```

```
Out[5]:
```

	machine_id	current_A	vibration_ms2	state
0	1	0.073	1.029	MACHINE ON (Idle)
1	2	0.039	0.301	MACHINE OFF
2	5	0.129	1.307	MACHINE ON + WORKING
3	1	0.030	0.010	MACHINE OFF
4	5	0.050	0.744	MACHINE OFF

```
In [6]: df.tail()
```

```
Out[6]:
```

	machine_id	current_A	vibration_ms2	state
5995	1	0.112	1.496	MACHINE ON (Idle)
5996	1	0.096	0.780	MACHINE ON (Idle)
5997	1	0.106	0.690	MACHINE ON (Idle)
5998	3	0.035	0.308	MACHINE OFF
5999	1	0.075	0.654	MACHINE ON (Idle)

```
In [7]: df.dtypes
```

```
Out[7]:
```

	0
machine_id	int64
current_A	float64
vibration_ms2	float64
state	object

dtype: object

```
In [9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   machine_id      6000 non-null   int64
1   current_A       6000 non-null   float64
2   vibration_ms2   6000 non-null   float64
3   state           6000 non-null   object
dtypes: float64(2), int64(1), object(1)
memory usage: 187.6+ KB
```

```
In [10]: df.isnull().sum()
```

```
Out[10]:
```

	0
machine_id	0
current_A	0
vibration_ms2	0
state	0

dtype: int64

```
In [12]: df.describe()
```

```
Out[12]:
```

	machine_id	current_A	vibration_ms2
count	6000.000000	6000.000000	6000.000000
mean	2.999167	0.088676	1.375754
std	1.415450	0.045768	1.685470
min	1.000000	0.015000	0.010000
25%	2.000000	0.043000	0.335000
50%	3.000000	0.092000	0.841000
75%	4.000000	0.121000	1.346000
max	5.000000	0.260000	9.500000

```
In [13]: df["state"].value_counts()
```

```
Out[13]:
```

	count
state	
MACHINE ON (Idle)	2143
MACHINE OFF	2084
MACHINE ON + WORKING	1773

dtype: int64

```
In [14]: df["state"].value_counts(normalize=True) * 100
```

```
Out[14]:
```

	proportion
state	
MACHINE ON (Idle)	35.716667
MACHINE OFF	34.733333
MACHINE ON + WORKING	29.550000

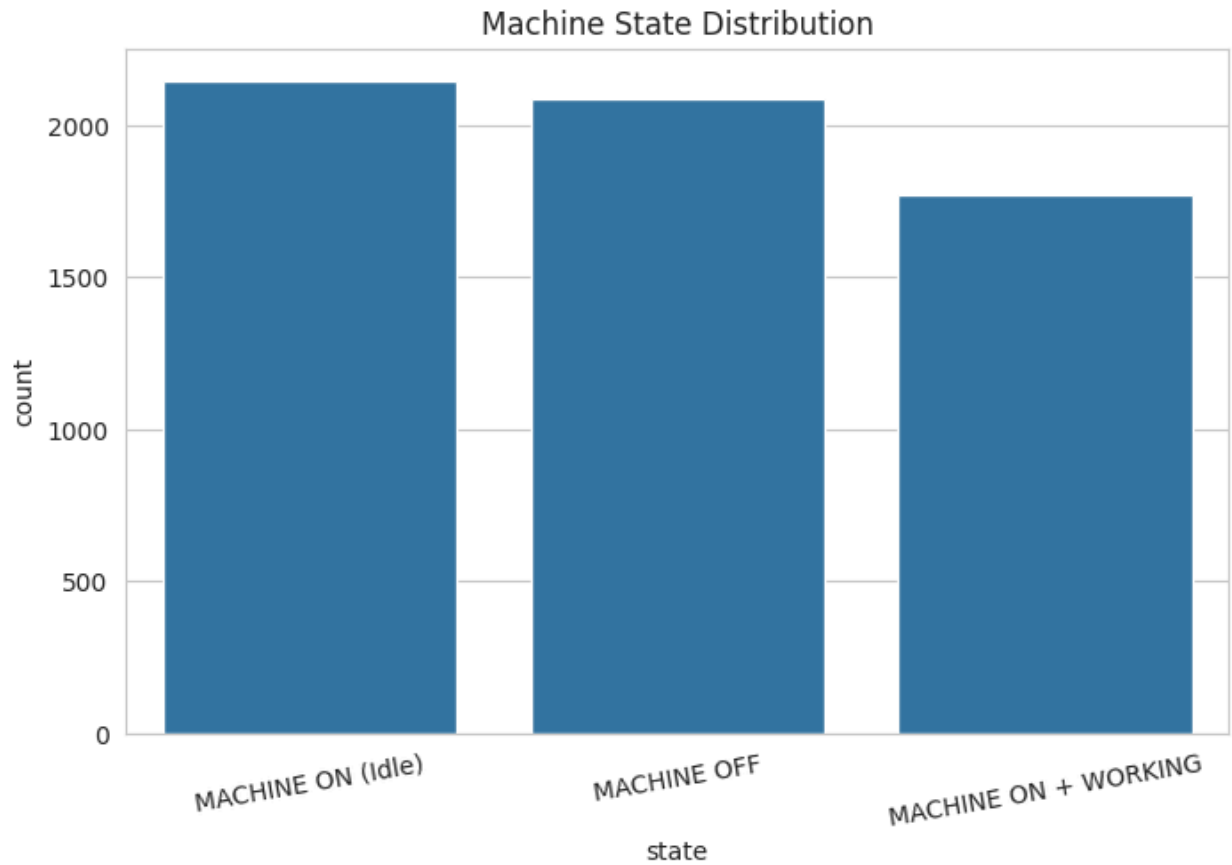
dtype: float64

```
In [15]: plt.figure(figsize=(8,5))

sns.countplot(data=df, x="state")

plt.title("Machine State Distribution")
plt.xticks(rotation=10)
```

```
plt.show()
```



The dataset contains comparable numbers of OFF, Idle, and Working states, which helps improve classification reliability across all machine conditions.

```
In [18]: df["machine_id"].unique()
```

```
Out[18]: array([1, 2, 5, 3, 4])
```

```
In [19]: df["machine_id"].value_counts()
```

```
Out[19]:
```

	count
--	-------

machine_id	
------------	--

2	1217
---	------

4	1206
---	------

5	1201
---	------

1	1198
---	------

3	1178
---	------

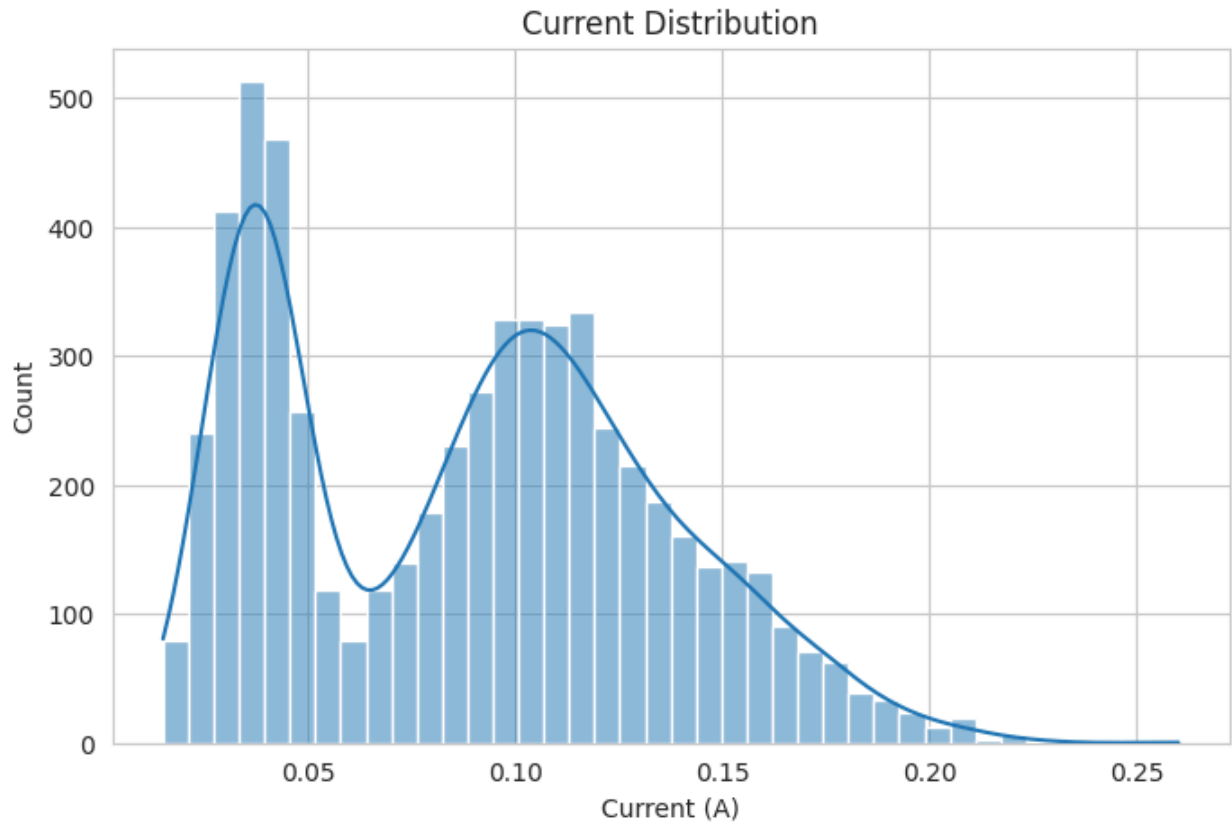
dtype: int64

Sensor readings are distributed across all five machines, enabling machine-wise behavioral analysis and improving dataset diversity.

```
In [21]: plt.figure(figsize=(8,5))

sns.histplot(df["current_A"], bins=40, kde=True)

plt.title("Current Distribution")
plt.xlabel("Current (A)")
plt.show()
```

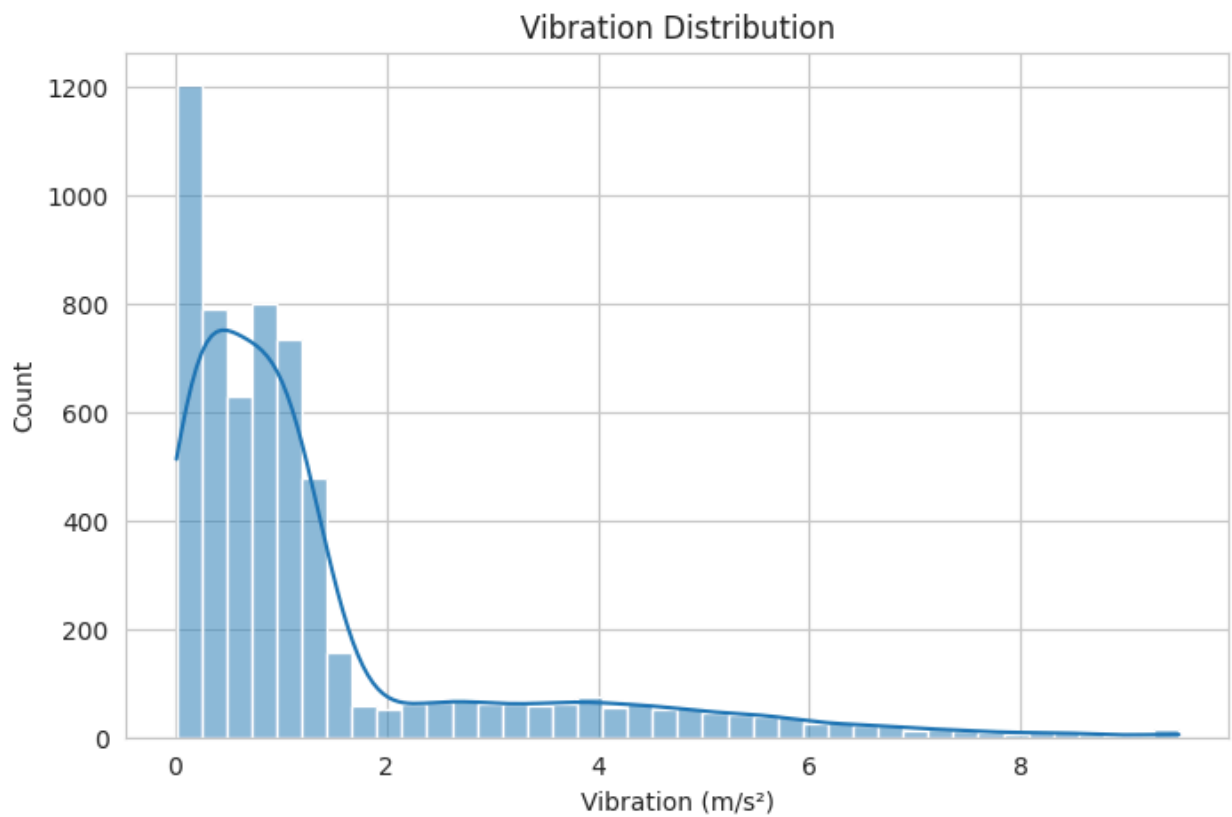


Current values show multiple operating regions corresponding to OFF, Idle, and Working conditions, along with slight overlap caused by sensor noise.

```
In [22]: plt.figure(figsize=(8,5))

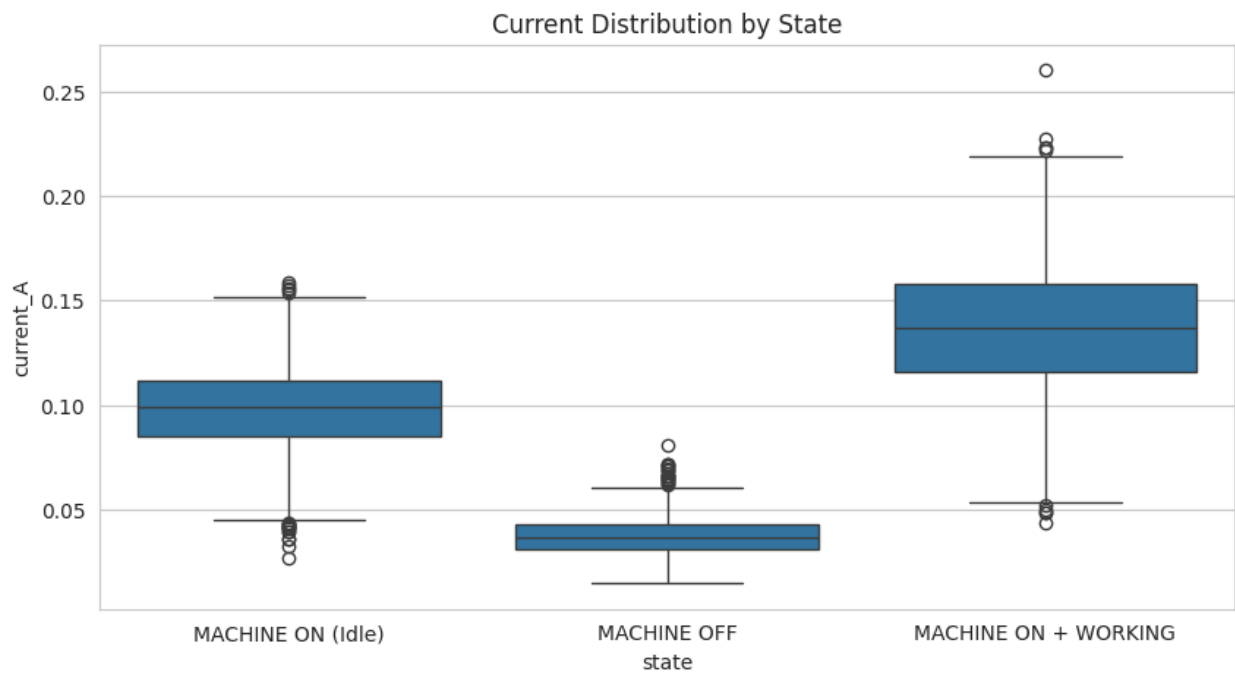
sns.histplot(df["vibration_ms2"], bins=40, kde=True)

plt.title("Vibration Distribution")
plt.xlabel("Vibration (m/s2)")
plt.show()
```

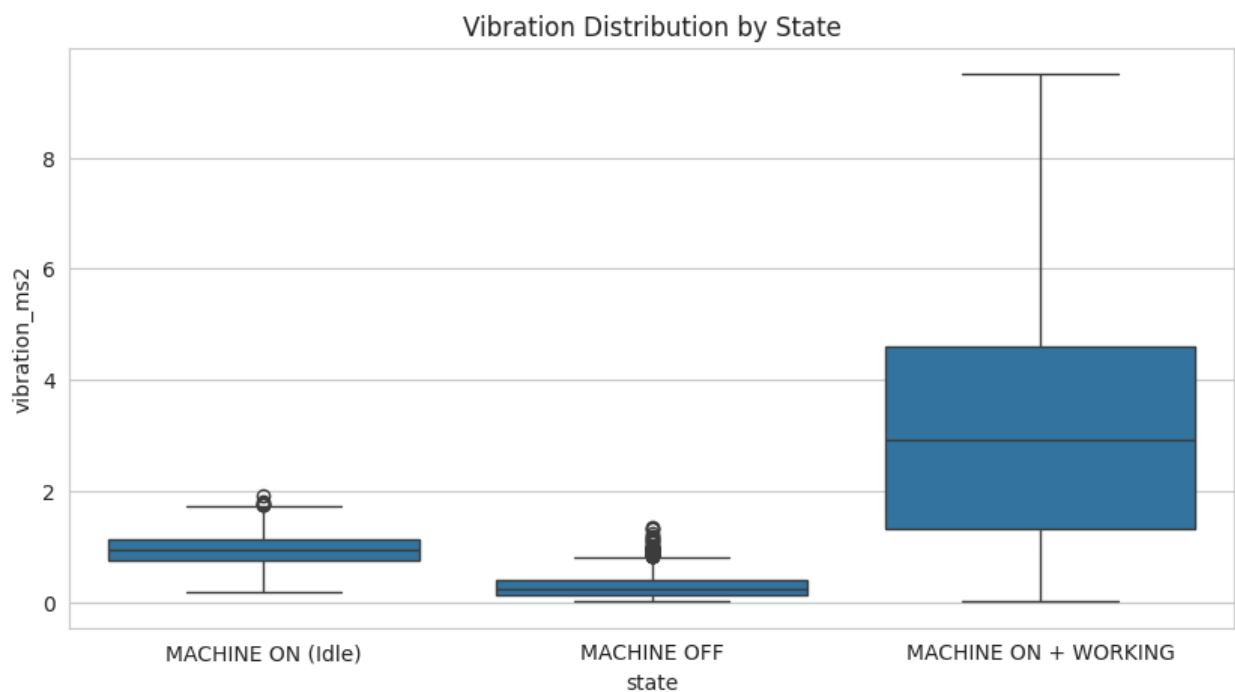
Vibration values exhibit a positively skewed distribution with several high-vibration samples representing heavy machining activity and industrial fluctuations.

```
In [23]: plt.figure(figsize=(10,5))  
  
sns.boxplot(data=df, x="state", y="current_A")  
  
plt.title("Current Distribution by State")  
plt.show()
```



Working states generally show higher current consumption, while overlap between Idle and Working states indicates realistic transitional machine behavior.

```
In [24]: plt.figure(figsize=(10,5))
sns.boxplot(data=df, x="state", y="vibration_ms2")
plt.title("Vibration Distribution by State")
plt.show()
```

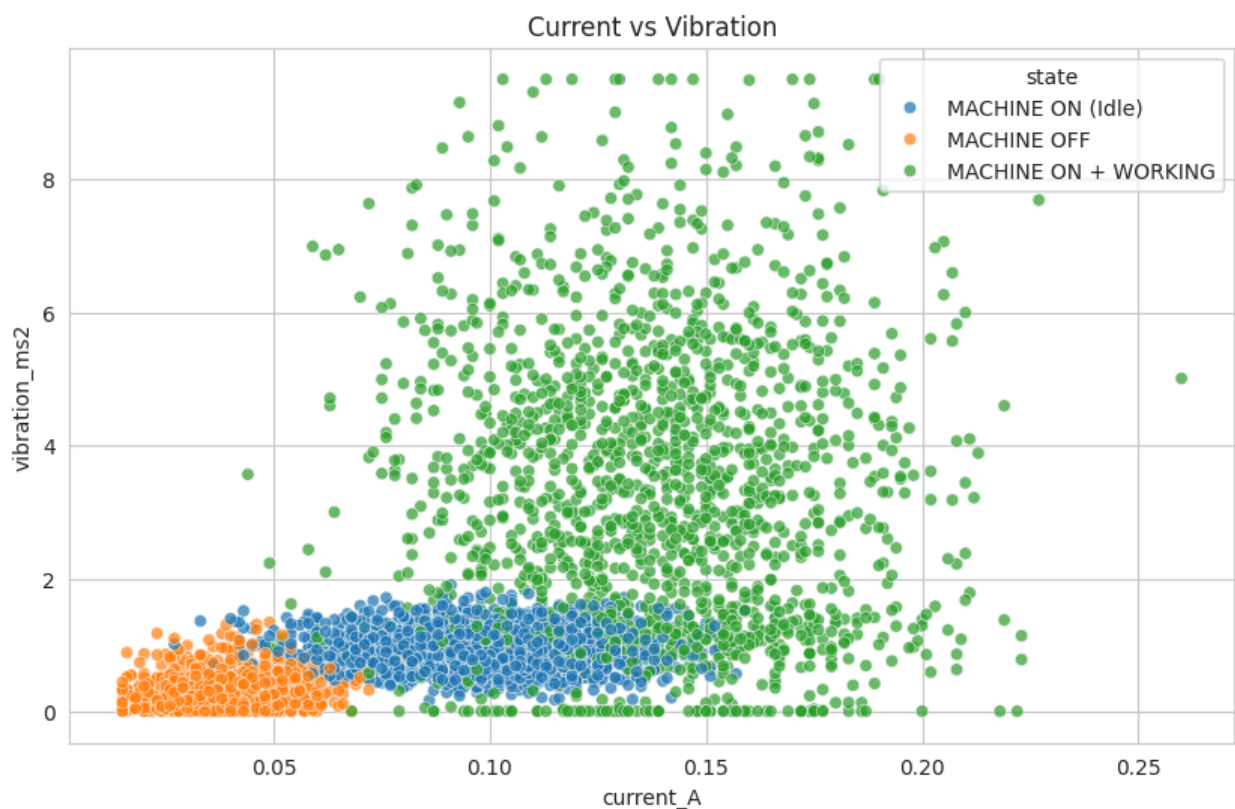


Working conditions exhibit significantly higher vibration levels and greater variability, while overlap near threshold regions reflects realistic sensor uncertainty.

```
In [25]: plt.figure(figsize=(10,6))

sns.scatterplot(
    data=df,
    x="current_A",
    y="vibration_ms2",
    hue="state",
    alpha=0.7
)

plt.title("Current vs Vibration")
plt.show()
```



The scatter plot reveals visible clustering between machine states while also showing overlapping boundary regions, demonstrating realistic industrial sensor behavior.

```
In [26]: from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df["state_encoded"] = le.fit_transform(df["state"])
```

```
In [27]: corr = df.corr(numeric_only=True)
```

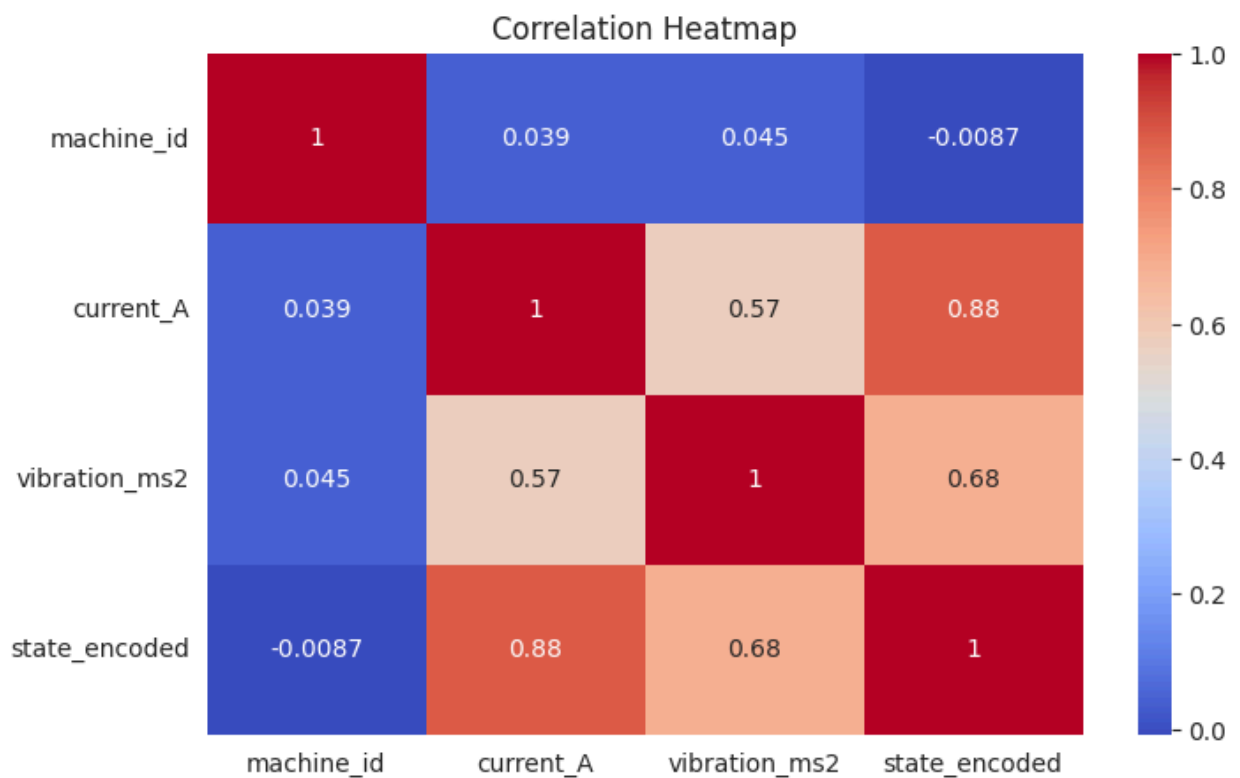
```
corr
```

```
Out[27]:
```

	machine_id	current_A	vibration_ms2	state_encoded
machine_id	1.000000	0.038953	0.044558	-0.008722
current_A	0.038953	1.000000	0.572523	0.877962
vibration_ms2	0.044558	0.572523	1.000000	0.681591
state_encoded	-0.008722	0.877962	0.681591	1.000000

Current and vibration show positive correlation with machine operational states, whereas machine ID has minimal influence on classification outcomes

```
In [28]: plt.figure(figsize=(8,5))  
  
sns.heatmap(corr, annot=True, cmap="coolwarm")  
  
plt.title("Correlation Heatmap")  
plt.show()
```



The heatmap confirms strong relationships between sensor readings and machine states, supporting the suitability of the dataset for predictive modeling.

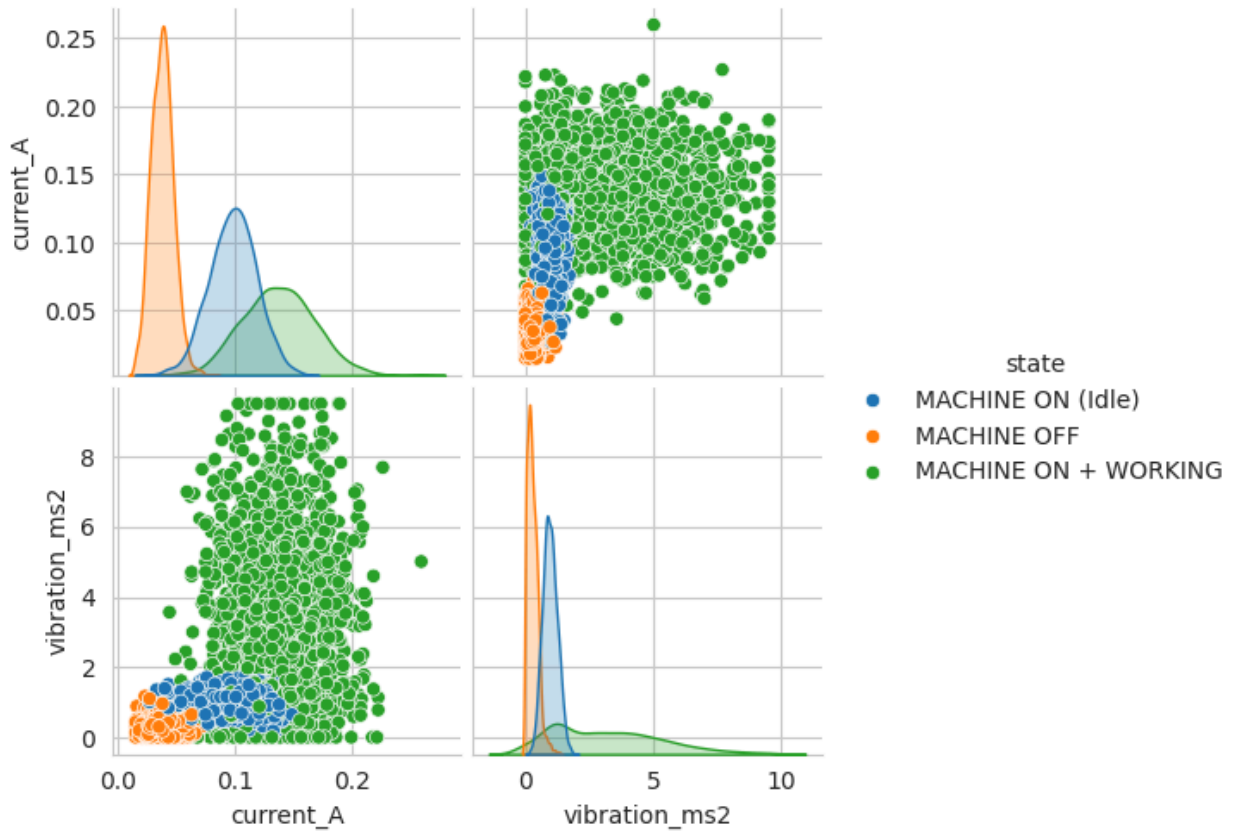
```
In [29]: sns.pairplot(  
    df,
```

```

hue="state",
vars=["current_A", "vibration_ms2"]
)

plt.show()

```



Pairwise visualization shows partial class separation along with noisy overlaps, indicating that the classification problem is realistic rather than perfectly separable.

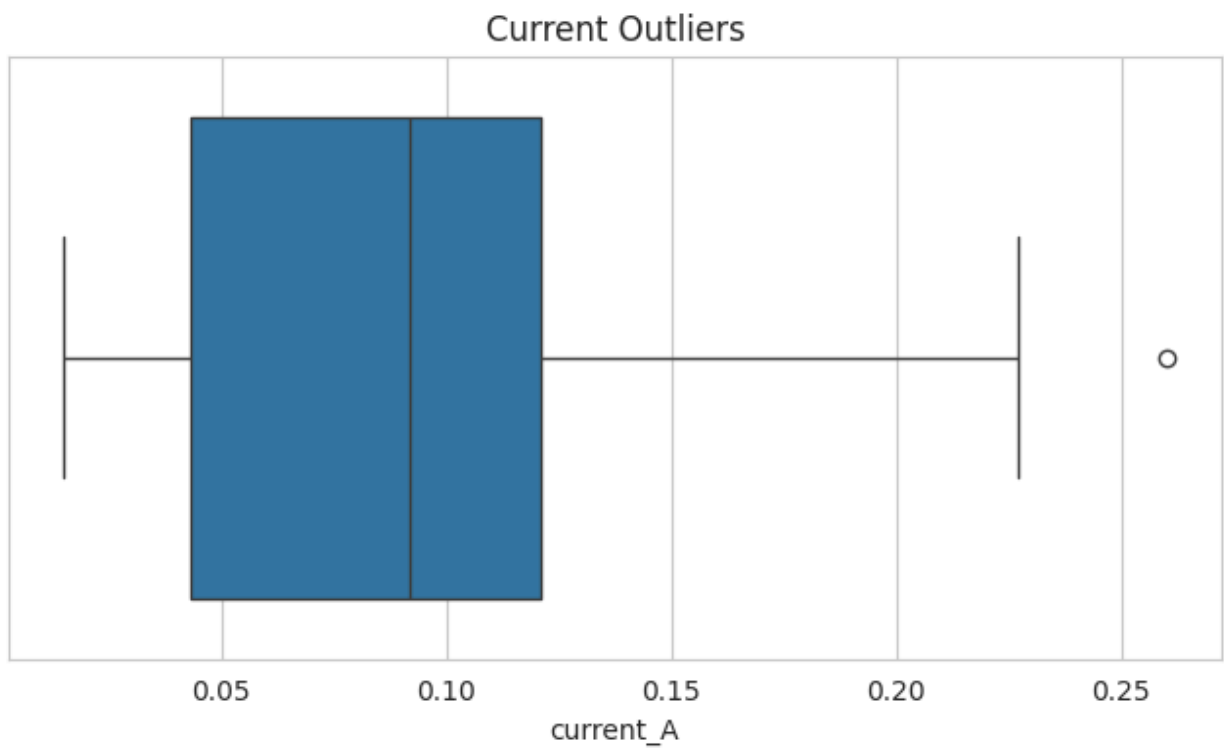
```

In [30]: plt.figure(figsize=(8,4))

sns.boxplot(x=df["current_A"])

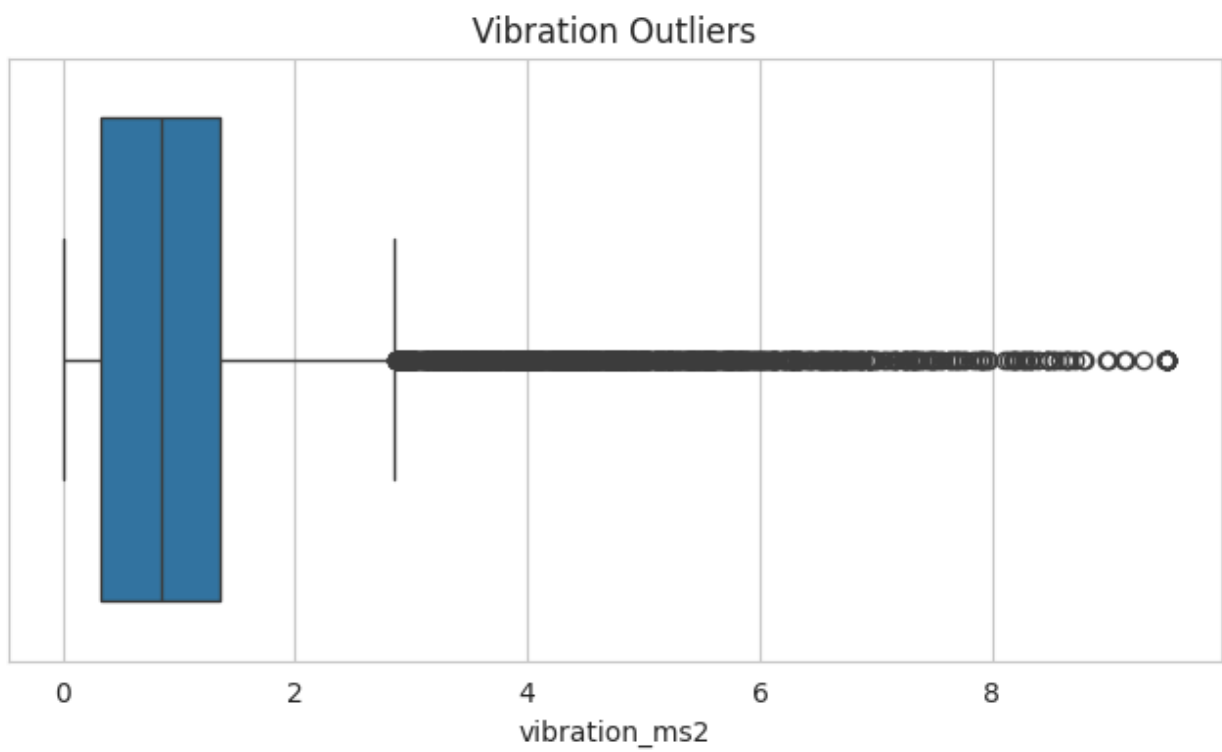
plt.title("Current Outliers")
plt.show()

```



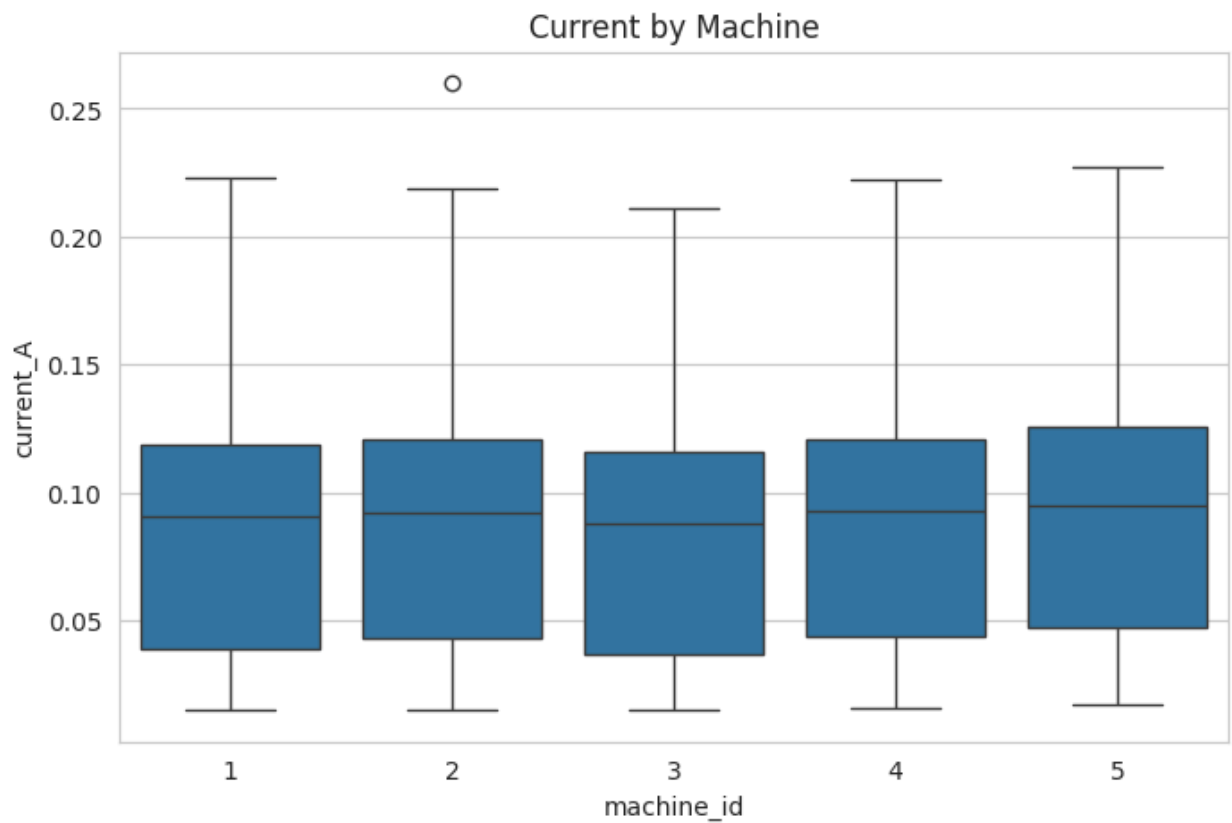
A small number of current outliers are present, representing occasional electrical fluctuations and transient industrial operating conditions.

```
In [32]: plt.figure(figsize=(8,4))  
  
sns.boxplot(x=df["vibration_ms2"])  
  
plt.title("Vibration Outliers")  
plt.show()
```



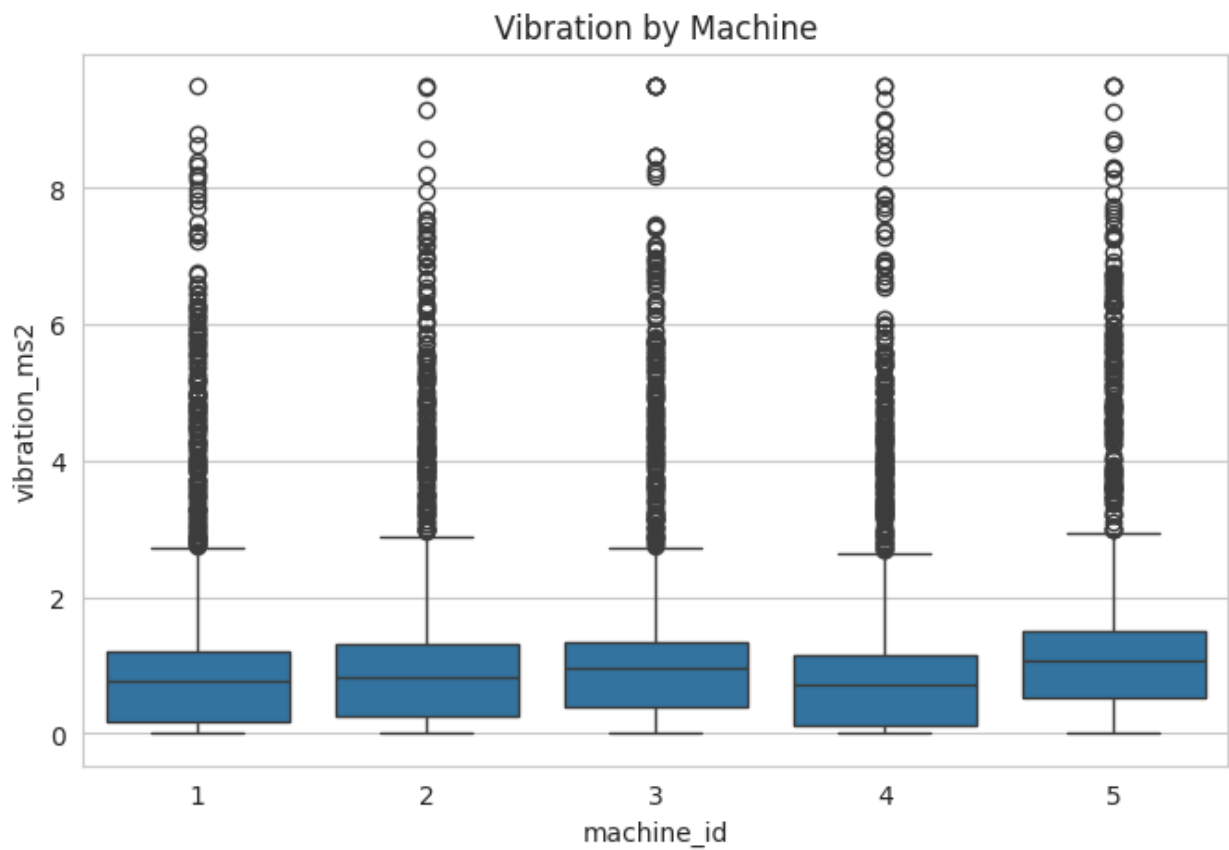
High-vibration outliers correspond to intense machining activity and simulated abnormal vibration spikes commonly observed in industrial environments.

```
In [33]: plt.figure(figsize=(8,5))  
  
sns.boxplot(data=df, x="machine_id", y="current_A")  
  
plt.title("Current by Machine")  
plt.show()
```



Different machines exhibit slight variations in current consumption patterns, reflecting realistic machine-specific operational behavior.

```
In [34]: plt.figure(figsize=(8,5))  
  
sns.boxplot(data=df, x="machine_id", y="vibration_ms2")  
  
plt.title("Vibration by Machine")  
plt.show()
```

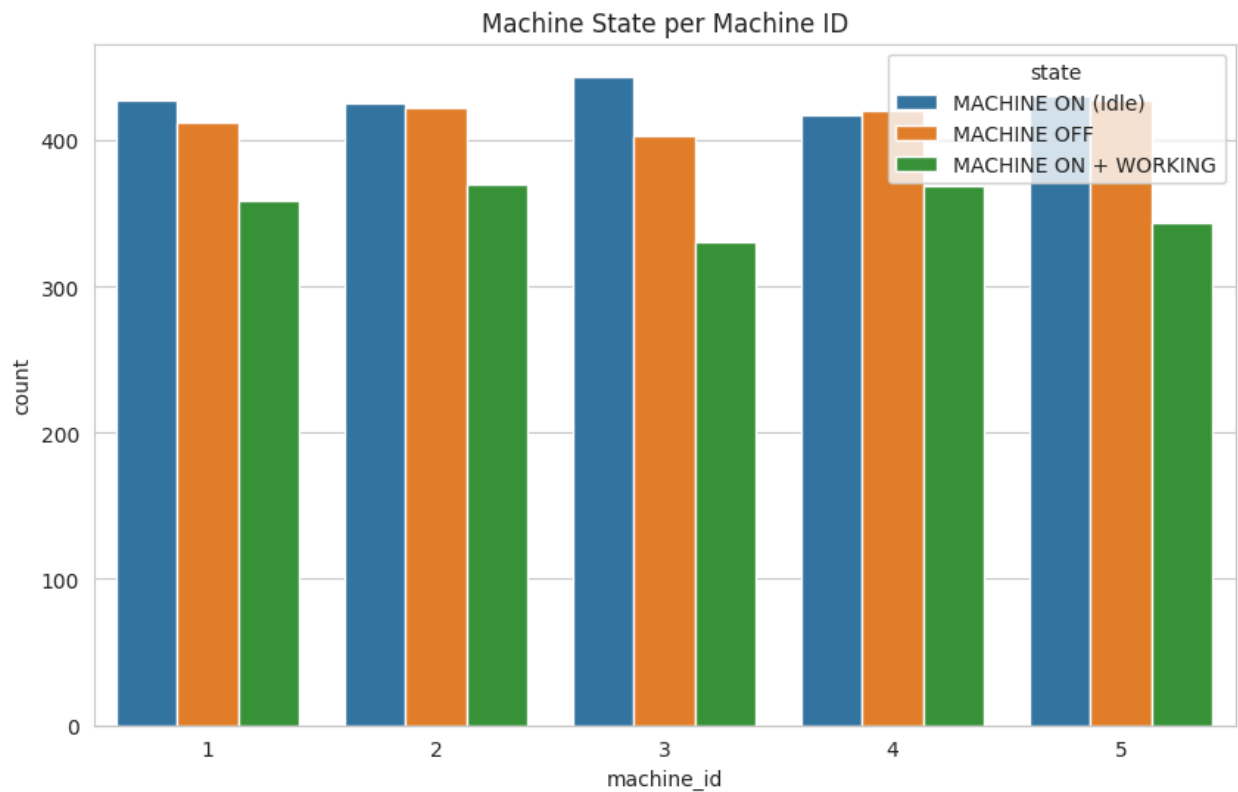



Machine-wise vibration distributions indicate that some machines produce higher vibration variability, simulating differences in machine condition and stability.

```
In [35]: plt.figure(figsize=(10,6))

sns.countplot(data=df, x="machine_id", hue="state")

plt.title("Machine State per Machine ID")
plt.show()
```



All machines contain a mixture of OFF, Idle, and Working states, suggesting that the dataset captures operational variability across multiple CNC machines and supports generalized model learning.

Final EDA Observation

Exploratory Data Analysis shows that the dataset contains distinguishable patterns between MACHINE OFF, MACHINE ON (Idle), and MACHINE ON + WORKING states based on current and vibration readings. The distributions of sensor values contain overlapping regions and outlier values, indicating non-linear separation between operational states.

Correlation analysis indicates that current and vibration features have strong association with machine state classification, whereas machine ID has minimal direct correlation with the target variable. Scatter plots, boxplots, and distribution analysis further demonstrate variations in sensor behavior across different machine conditions and machine IDs.

Based on the observed feature distributions and relationships, the dataset is suitable for training and evaluating multiple machine

learning classification models for CNC machine state prediction.

Model Training

Random Forest

```
In [36]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from sklearn.ensemble import RandomForestClassifier

import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [37]: X = df[["machine_id", "current_A", "vibration_ms2"]]
y = df["state"]
```

```
In [38]: le = LabelEncoder()

y_encoded = le.fit_transform(y)
```

```
In [39]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
```

```
In [40]: rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

```
Out[40]: ▼ RandomForestClassifier ⓘ ?
RandomForestClassifier(random_state=42)
```

```
In [41]: y_pred = rf_model.predict(X_test)
```

```
In [42]: accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")
```

Accuracy: 93.92 %

```
In [43]: print(classification_report(
    y_test,
    y_pred,
    target_names=le.classes_
))
```

	precision	recall	f1-score	support
MACHINE OFF	0.99	1.00	0.99	417
MACHINE ON (Idle)	0.89	0.94	0.92	428
MACHINE ON + WORKING	0.94	0.87	0.90	355
accuracy			0.94	1200
macro avg	0.94	0.94	0.94	1200
weighted avg	0.94	0.94	0.94	1200

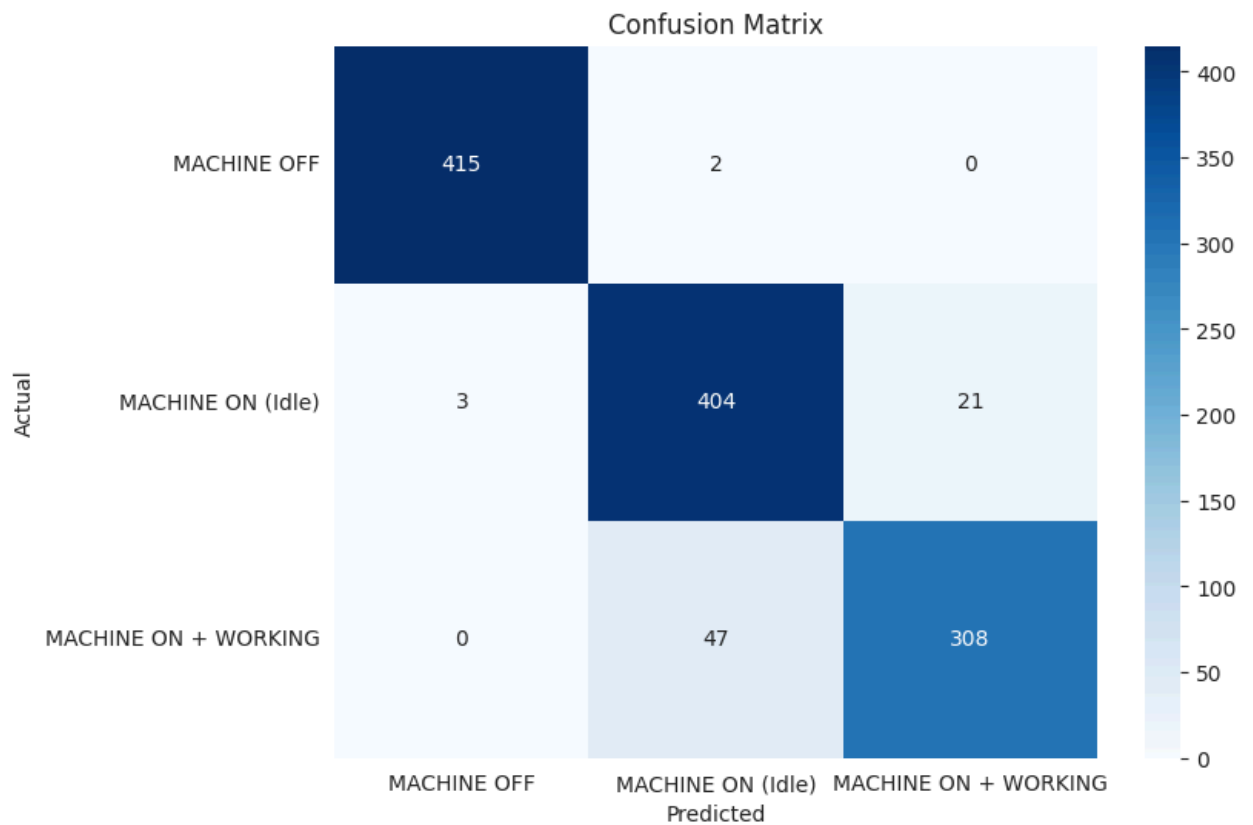
```
In [44]: cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=le.classes_,
    yticklabels=le.classes_
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

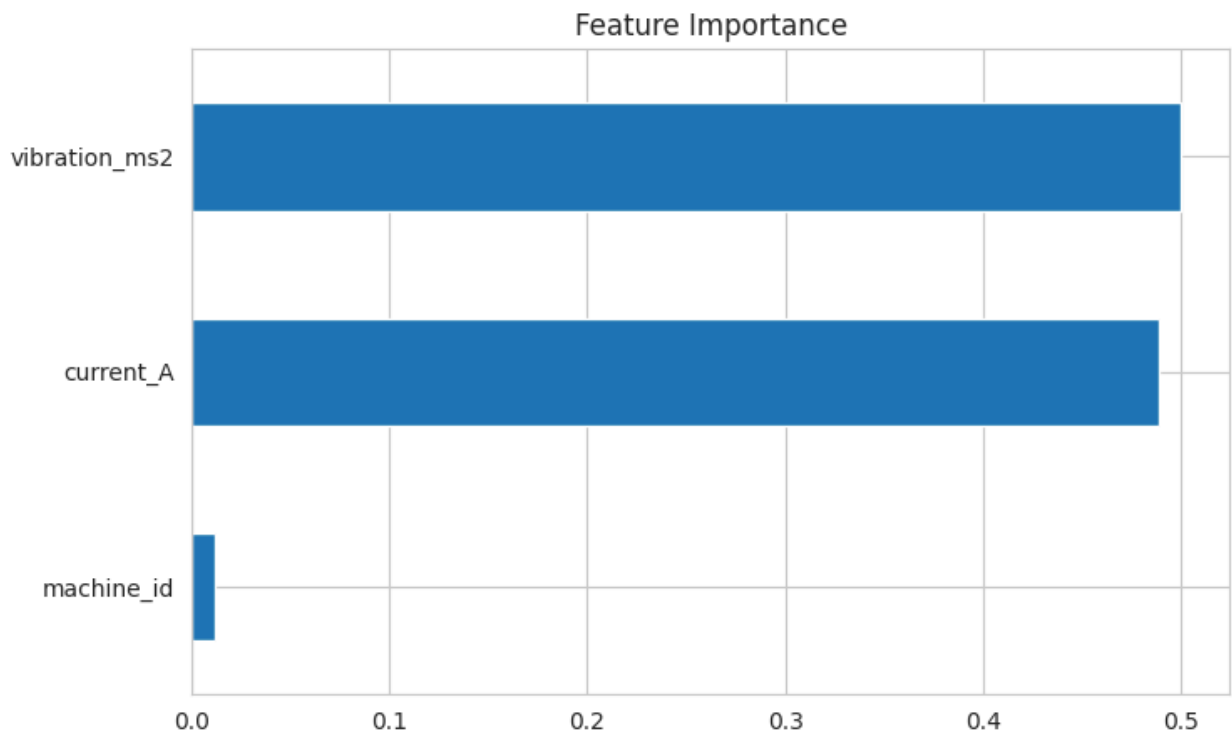


```
In [45]: import pandas as pd

importance = pd.Series(
    rf_model.feature_importances_,
    index=X.columns
)

importance.sort_values().plot(
    kind="barh",
    figsize=(8,5),
    title="Feature Importance"
)

plt.show()
```



KNN

```
In [49]: from sklearn.neighbors import KNeighborsClassifier
```

```
In [50]: knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
```

```
Out[50]: ▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier()
```

```
In [51]: y_pred_knn = knn_model.predict(X_test)
```

```
In [52]: knn_accuracy = accuracy_score(y_test, y_pred_knn)
print("KNN Accuracy:",
      round(knn_accuracy * 100, 2), "%")
```

KNN Accuracy: 91.5 %

```
In [53]: print(classification_report(
    y_test,
    y_pred_knn,
    target_names=le.classes_
))
```

	precision	recall	f1-score	support
MACHINE OFF	0.99	0.97	0.98	417
MACHINE ON (Idle)	0.84	0.95	0.89	428
MACHINE ON + WORKING	0.94	0.81	0.87	355
accuracy			0.92	1200
macro avg	0.92	0.91	0.91	1200
weighted avg	0.92	0.92	0.91	1200

Decision Tree

```
In [54]: from sklearn.tree import DecisionTreeClassifier
```

```
In [55]: dt_model = DecisionTreeClassifier(
            random_state=42,
            max_depth=8
        )

dt_model.fit(X_train, y_train)
```

```
Out[55]: ▼ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=8, random_state=42)
```

```
In [56]: y_pred_dt = dt_model.predict(X_test)
```

```
In [57]: dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("Decision Tree Accuracy:",
      round(dt_accuracy * 100, 2), "%")
```

Decision Tree Accuracy: 94.17 %

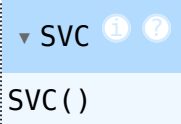
```
In [58]: print(classification_report(
            y_test,
            y_pred_dt,
            target_names=le.classes_
        ))
```

	precision	recall	f1-score	support
MACHINE OFF	1.00	0.99	0.99	417
MACHINE ON (Idle)	0.89	0.96	0.92	428
MACHINE ON + WORKING	0.95	0.87	0.91	355
accuracy			0.94	1200
macro avg	0.94	0.94	0.94	1200
weighted avg	0.94	0.94	0.94	1200

Support Vector Machine (SVM)

```
In [59]: from sklearn.svm import SVC
```

```
In [60]: svm_model = SVC(kernel="rbf")  
  
svm_model.fit(X_train, y_train)
```

```
Out[60]:  SVC()
```

```
In [61]: y_pred_svm = svm_model.predict(X_test)
```

```
In [62]: svm_accuracy = accuracy_score(y_test, y_pred_svm)  
  
print("SVM Accuracy:",  
      round(svm_accuracy * 100, 2), "%")
```

SVM Accuracy: 87.42 %

```
In [63]: print(classification_report(  
          y_test,  
          y_pred_svm,  
          target_names=le.classes_  
        ))
```

	precision	recall	f1-score	support
MACHINE OFF	0.90	0.97	0.93	417
MACHINE ON (Idle)	0.80	0.96	0.87	428
MACHINE ON + WORKING	1.00	0.66	0.80	355
accuracy			0.87	1200
macro avg	0.90	0.86	0.87	1200
weighted avg	0.89	0.87	0.87	1200

Final Comparison Table

```
In [69]: comparison = pd.DataFrame({  
        "Model": [  
            "Random Forest",  
            "KNN",  
            "Decision Tree",  
            "SVM"  
        ],  
        "Accuracy": [  
            accuracy,  
            knn_accuracy,
```



```

        dt_accuracy,
        svm_accuracy,
    ]
})

comparison["Accuracy"] = comparison["Accuracy"] * 100

comparison

```

Out[69]:

	Model	Accuracy
0	Random Forest	93.916667
1	KNN	91.500000
2	Decision Tree	94.166667
3	SVM	87.416667

```

In [68]: plt.figure(figsize=(10,5))

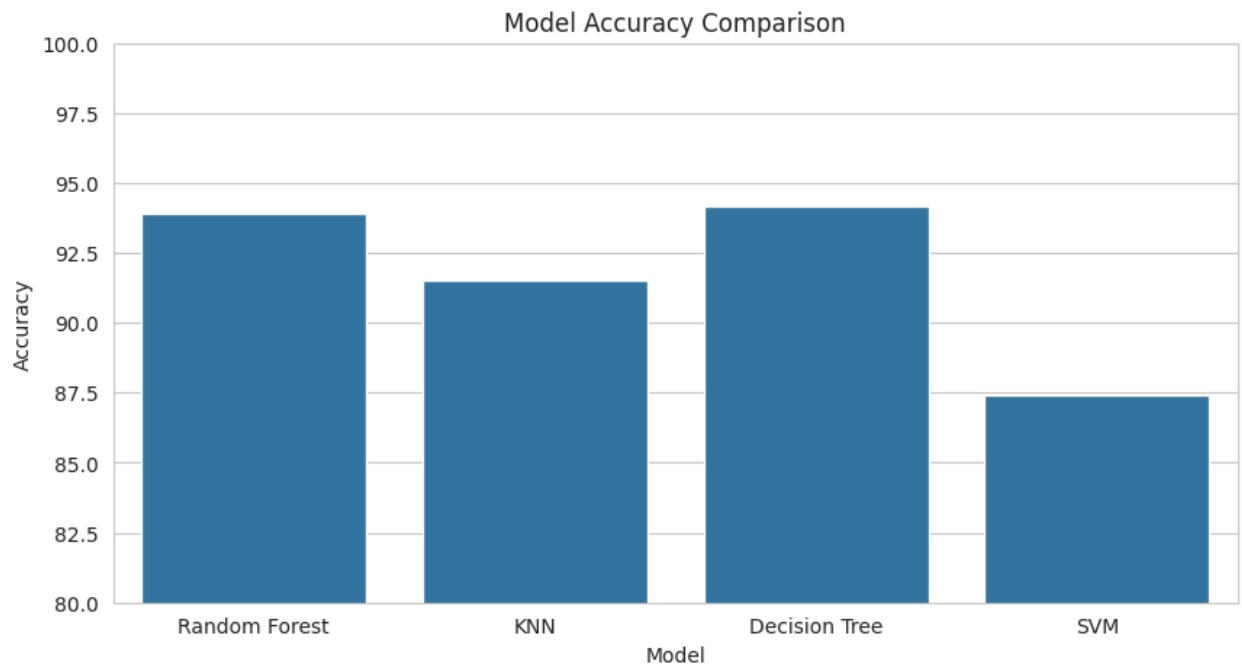
sns.barplot(
    data=comparison,
    x="Model",
    y="Accuracy"
)

plt.title("Model Accuracy Comparison")

plt.ylim(80, 100)

plt.show()

```



Decision Tree achieved the highest accuracy on the testing dataset. However, Random Forest was selected as the final deployment model because ensemble learning provides better robustness, reduced overfitting, and improved generalization for noisy industrial sensor data.